

# Tanking the Stream: Residual Emergent Distillation of Closed Open-Weight Transformers into Hilbert Space Perceptrons

**Author:** Manus AI

**Date:** July 5, 2026

## Abstract

The proliferation of “closed open-weight” models — large language models (LLMs) whose parameters are publicly accessible but whose training data, intermediate checkpoints, and architectural rationale remain proprietary — presents a unique challenge for knowledge transfer and model compression. While traditional black-box distillation relies solely on input-output pairs, open-weight access permits the direct observation of internal computational dynamics. In this white paper, we formalize the **Residual Emergent Method**, a technique for “tanking” (harvesting and isolating) the information flowing through the transformer’s residual stream. By extracting the emergent, layer-wise contributions of Feed-Forward Networks (FFNs) and mapping them via Sparse Autoencoders (SAEs), we establish a protocol for distilling these proprietary representations into compact, convex Hilbert Space Perceptron (HSP) modules. We conclude with a rigorous audit of this methodology, evaluating its mathematical validity, empirical limitations, and the complex legal landscape governing derivative model licensing.

---

## 1. Introduction

The landscape of foundation models is increasingly dominated by “closed open-weight” architectures, such as Meta’s Llama 3, Google’s Gemma, and Mistral’s larger variants [1] [2]. These models grant users full access to the inference graph and weight matrices, yet the massive datasets and computational pipelines used to train them remain hidden. Consequently, compressing these models or transferring their specific capabilities to custom architectures requires methods that go beyond traditional knowledge distillation.

Standard knowledge distillation minimizes the Kullback-Leibler (KL) divergence between the final output logits of a teacher and a student model [3]. However, this approach treats the teacher as a black box, ignoring the rich hierarchy of intermediate representations. Mechanistic interpretability research has demonstrated that the transformer architecture operates as a series of additive updates to a central communication channel known as the *residual stream* [4].

The **Residual Emergent Method** exploits this architectural property. By executing the closed open-weight model over a diverse proxy corpus, we can “tank” (harvest and store) the intermediate residual stream vectors. Because the FFNs in these models often exhibit high polysemanticity (neurons responding to multiple unrelated features) [5], we apply Sparse Autoencoders (SAEs) to disentangle these representations into monosemantic, emergent features [6]. Finally, we distill these isolated feature mappings into Hilbert Space Perceptrons (HSPs), achieving massive parameter reduction while inheriting the teacher’s internal logic.

---

## 2. Mathematical Formulation of the Residual Stream

To formalize the extraction process, we must define the computational flow of the transformer architecture.

### 2.1 The Additive Residual Stream

Let  $\mathbf{x}^0 \in \mathbb{R}^{B \times S \times d}$  denote the embedded input sequence, where  $B$  is batch size,  $S$  is sequence length, and  $d$  is the residual stream dimension. The transformer processes this input through  $L$  identical layers. According to the framework established by Elhage et al. [4], the residual stream at layer  $l$  is the sum of all previous layer outputs:

$$\mathbf{x}^l = \mathbf{x}^{l-1} + \text{Attn}^l(\text{LN}(\mathbf{x}^{l-1})) + \text{FFN}^l(\text{LN}(\mathbf{x}^{l-1}))$$

By unrolling this recurrence, the final residual stream vector  $\mathbf{x}^L$  is strictly additive:

$$\mathbf{x}^L = \mathbf{x}^0 + \sum_{i=1}^L \Delta_{\text{Attn}}^i + \sum_{i=1}^L \Delta_{\text{FFN}}^i$$

Where  $\Delta_{\text{Attn}}^i$  and  $\Delta_{\text{FFN}}^i$  represent the emergent updates contributed by the attention mechanism and the Feed-Forward Network at layer  $i$ , respectively.

### 2.2 Isolating the Target Knowledge

When compressing a transformer, the FFN sublayers are the primary target, as they typically account for two-thirds of the total parameter count [7]. To “tank” the knowledge of a specific FFN sublayer at layer  $l$ , we isolate its input-output mapping during inference:

1. **Input State:**  $\mathbf{h}_{in}^l = \text{LN}(\mathbf{x}^{l-1})$
2. **Emergent Update:**  $\mathbf{h}_{out}^l = \Delta_{\text{FFN}}^l = \text{FFN}^l(\mathbf{h}_{in}^l)$

This pair  $(\mathbf{h}_{in}^l, \mathbf{h}_{out}^l)$  encapsulates the precise, localized knowledge the teacher model applies at depth  $l$ .

---

### 3. The Residual Emergent Distillation Protocol

With the mathematical target defined, we outline the end-to-end protocol for transferring this knowledge into a compact HSP module.

#### 3.1 Step 1: Activation Harvesting (“Tanking”)

We require a proxy dataset  $\mathcal{D}_{proxy}$  that covers the domain of interest. We perform forward passes of the closed open-weight model  $M_{teacher}$  over  $\mathcal{D}_{proxy}$ . For a target layer  $l$ , we harvest the intermediate activations:

$$\mathcal{A}^l = \{(\mathbf{h}_{in,k}^l, \mathbf{h}_{out,k}^l)\}_{k=1}^K$$

Where  $K$  is the total number of tokens processed. This dataset  $\mathcal{A}^l$  represents the “tanked” residual stream dynamics.

#### 3.2 Step 2: Disentangling Emergent Features via SAE

The FFN output  $\mathbf{h}_{out}^l$  resides in the residual stream dimension  $d$ . According to the superposition hypothesis [5], the model stores  $N \gg d$  features as nearly orthogonal vectors within this space. To distill this effectively, we must disentangle these polysemantic representations.

We train a Sparse Autoencoder (SAE) on the harvested updates  $\mathbf{h}_{out}^l$  [6]. The SAE consists of an encoder matrix  $\mathbf{W}_{enc} \in \mathbb{R}^{d \times N_{SAE}}$  and a decoder matrix  $\mathbf{W}_{dec} \in \mathbb{R}^{N_{SAE} \times d}$ :

$$\mathbf{f}^l = \text{ReLU}(\mathbf{h}_{out}^l \mathbf{W}_{enc} + \mathbf{b}_{enc})$$

$$\hat{\mathbf{h}}_{out}^l = \mathbf{f}^l \mathbf{W}_{dec} + \mathbf{b}_{dec}$$

The SAE is trained with an  $L_1$  sparsity penalty on the feature activations  $\mathbf{f}^l$ . Once trained, the sparse vector  $\mathbf{f}^l$  represents the true, disentangled emergent concepts activated by the FFN at layer  $l$ .

#### 3.3 Step 3: Spectral Projection to HSP

We define a target Hilbert Space Perceptron  $\mathcal{H}^l$  operating over an orthonormal basis  $\Phi$ . The goal is to map the FFN input  $\mathbf{h}_{in}^l$  directly to the disentangled emergent features  $\mathbf{f}^l$ , bypassing the massive SwiGLU FFN matrices entirely.

Using the spectral distillation methodology, we project the harvested input states onto the Hilbert basis:

$$\mathbf{Z}^l = \Phi(\mathbf{H}_{in}^l)$$

We then initialize the HSP spectral weights  $\mathbf{W}_{HSP}$  via ordinary least squares against the SAE feature targets:

$$\mathbf{W}_{HSP}^{(0)} = ((\mathbf{Z}^l)^T \mathbf{Z}^l)^{-1} (\mathbf{Z}^l)^T \mathbf{F}^l$$

### 3.4 Step 4: Convex Fine-Tuning

Finally, the HSP is fine-tuned to minimize the Mean Squared Error (MSE) between its predictions and the SAE-extracted emergent features:

$$\mathcal{L}_{HSP} = \frac{1}{K} \sum_{k=1}^K \|\mathcal{H}^l(\mathbf{h}_{in,k}^l) - \mathbf{f}_k^l\|_2^2$$

Because the HSP operates over an orthonormal basis, this loss landscape is strictly convex, guaranteeing convergence to the global minimum without the vanishing gradient issues common in deep MLPs.

---

## 4. Audit of the Residual Emergent Method

To validate this approach, we must conduct a rigorous audit across three dimensions: mathematical validity, empirical limitations, and legal compliance.

### 4.1 Mathematical and Architectural Validity

The fundamental premise—that the residual stream can be decomposed into additive, layer-wise contributions—is architecturally sound and supported by extensive mechanistic interpretability literature [4] [6].

#### Strengths:

- By targeting  $\Delta_{FFN}^l$  rather than final logits, the method avoids the “information bottleneck” of the unembedding matrix, capturing the full dimensionality of the model’s internal reasoning.
- The use of SAEs to map from the residual dimension  $d$  to the feature dimension  $N_{SAE}$  correctly addresses the superposition problem, ensuring the HSP learns distinct concepts rather than entangled noise [5].

## Vulnerabilities:

- **Distributional Shift:** The harvested activations  $\mathcal{A}^l$  are highly dependent on the proxy dataset  $\mathcal{D}_{proxy}$ . If the proxy data does not trigger the full spectrum of emergent features, the resulting HSP will suffer from catastrophic failure in out-of-distribution inference.

## 4.2 Empirical and Computational Limitations

While the loss landscape of the HSP is convex, the pipeline relies on the upstream SAE, which is non-convex and notoriously difficult to train perfectly.

| Component               | Computational Cost                       | Convexity Guarantee |
|-------------------------|------------------------------------------|---------------------|
| Forward Pass Harvesting | High (Inference on large model)          | N/A                 |
| SAE Training            | Very High (Requires massive batch sizes) | No                  |
| HSP Initialization      | Low (Closed-form linear algebra)         | Yes                 |
| HSP Fine-Tuning         | Low (Rapid convergence)                  | Yes                 |

If the SAE fails to achieve a low reconstruction error (i.e.,  $\hat{\mathbf{h}}_{out}^l \not\approx \mathbf{h}_{out}^l$ ), the HSP will perfectly learn an incorrect or incomplete target distribution.

## 4.3 Legal and Licensing Audit

The most critical bottleneck for the Residual Emergent Method is the legal framework governing closed open-weight models. The method inherently creates a “derivative model” based on the outputs and internal states of the teacher.

- **Meta Llama 3:** The Llama 3 Community License explicitly prohibits using the model, its outputs, or its internal representations to “train, improve, or build any other AI model” outside of the Llama ecosystem [1]. Applying this method to Llama 3 for a non-Llama target architecture constitutes a direct license violation.
- **Google Gemma:** The Gemma Terms of Use stipulate that any model trained on Gemma outputs is considered a Gemma derivative and is subject to the same restrictions [2].
- **Mistral:** The legality depends on the specific model. Mistral 7B is released under the permissive Apache 2.0 license, making it a legally viable target for this method. However,

larger models (e.g., Mistral Large) are governed by the restrictive Mistral AI Non-Production License (MNPL), which prohibits commercial use of derivatives [8].

**Audit Conclusion:** The Residual Emergent Method is mathematically elegant and architecturally viable for extreme compression. However, its application must be strictly restricted to models with permissive licenses (e.g., Apache 2.0, MIT) to avoid severe intellectual property violations.

---

## 5. Conclusion

“Tanking” the residual stream of closed open-weight models provides a high-fidelity pathway for knowledge extraction. By decomposing the transformer into additive updates, disentangling polysemantic features with Sparse Autoencoders, and projecting the results into convex Hilbert Space Perceptrons, we can achieve massive parameter compression. While the computational overhead of harvesting and SAE training is non-trivial, the primary barrier to adoption remains the restrictive licensing agreements that govern modern foundation models.

---

## References

- [1] Meta Platforms, Inc. “Llama 3 Community License Agreement.” GitHub, Mar 2026. Available: <https://github.com/meta-llama/llama3/blob/main/LICENSE>
- [2] Google. “Gemma Terms of Use.” Google AI for Developers. Available: <https://ai.google.dev/gemma/terms>
- [3] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv preprint arXiv:1503.02531, 2015.
- [4] N. Elhage et al., “A Mathematical Framework for Transformer Circuits,” Transformer Circuits Thread, 2021. Available: <https://transformer-circuits.pub/2021/framework/index.html>
- [5] N. Elhage et al., “Toy Models of Superposition,” Transformer Circuits Thread, 2022. Available: [https://transformer-circuits.pub/2022/toy\\_model/index.html](https://transformer-circuits.pub/2022/toy_model/index.html)
- [6] R. Huben et al., “Sparse Autoencoders Find Highly Interpretable Features in Language Models,” ICLR, 2024.
- [7] W. Wang et al., “Model compression and efficient inference for large language models: A survey,” arXiv preprint arXiv:2402.09748, 2024.
- [8] Mistral AI. “The Mistral AI Non-Production License.” Mistral AI Blog, May 2024. Available: <https://mistral.ai/news/mistral-ai-non-production-license-mnpl/>